

Reviewer Pack — Plethysm Multiplicity Inverse Proportional to Monotone Circu...

Entry 2e8cd8937a3c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 16:08:23 UTC

Plethysm Multiplicity Inverse Proportional to Monotone Circuit Size for Permanent

Entry ID: 2e8cd8937a3c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 16:08:23 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups (Plethysm Coefficients) **Field B** (complexity object): Monotone Circuit Size for Permanent Polynomial

Statement:

For a random 3-SAT instance Φ with n variables, let $\mu(\Phi)$ denote the plethysm coefficient of the symmetric power $S^k(\text{perm}_n)$ decomposed into irreducible GL_n -modules. The monotone circuit size $M(\Phi)$ satisfies $M(\Phi) \geq \Omega(\mu(\Phi)^{-1} n^2 \log n)$.

Rationale (proposer's reasoning):

Plethysm coefficients encode symmetry constraints of tensor powers, which may impose structural limits on monotone circuits. Symmetric decomposition patterns could reveal inherent complexity barriers via representation-theoretic invariants.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 816f1c2f23c4954f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    k = n // 2

    # Generate a random 3-SAT instance
    m = random.randint(1, 5 * n)
    C = [[random.choice([0, 1]) for _ in range(n)] for _ in range(m)]

    # Compute the symmetric power  $S^k(C)$ 
    def matrix_power(A, k):
        result = [[0] * n for _ in range(n)]
        for i in range(n):
            result[i][i] = 1
        for _ in range(k):
            result = matrix_multiply(result, A)
        return result

    def matrix_multiply(A, B):
        result = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for l in range(n):
                    result[i][j] += A[i][l] * B[l][j]
        return result

    S_k_C = matrix_power(C, k)

    # Decompose into irreducible  $GL_n$ -modules via Young tableaux algorithms
    # This is a placeholder for the actual decomposition algorithm
    # For simplicity, we assume  $\mu(\emptyset)$  is the multiplicity of the trivial representation
    def plethysm_coefficient(S_k_C):
        return sum(sum(row) for row in S_k_C)

     $\mu_{\emptyset}$  = plethysm_coefficient(S_k_C)
```

```

# Measure monotone circuit size via dynamic programming on the clause matrix
def monotone_circuit_size(C):
    m, n = len(C), len(C[0])
    dp = [[float('inf')] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = 0
    for i in range(1, m + 1):
        for j in range(n + 1):
            if C[i - 1][j - 1] == 1:
                dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + 1
            else:
                dp[i][j] = min(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]

M_Φ = monotone_circuit_size(C)

# Verify if  $M(\Phi) \geq c \cdot \mu(\Phi)^{-1} n^2 \log n$  for some  $c > 0$ 
c = 1.0
if μ_Φ == 0:
    return {
        "metric_name": "monotone_circuit_size",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "plethysm_coefficient_is_zero"
    }

if M_Φ >= c * μ_Φ**(-1) * n**2 * math.log(n):
    return {
        "metric_name": "monotone_circuit_size",
        "metric_value": M_Φ,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    return {
        "metric_name": "monotone_circuit_size",
        "metric_value": M_Φ,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f" $M(\Phi) < c \cdot \mu(\Phi)^{-1} n^2 \log n$ "
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)

```

```

std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'monotone_circuit_size', 'metric_value': 29, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'monotone_circuit_size', 'metric_value': 47, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'monotone_circuit_size', 'metric_value': 43, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'monotone_circuit_size', 'metric_value': 32, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'monotone_circuit_size', 'metric_value': 18, 'instances_tested': 1, 'conjecture_holds': True}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_49c604c9.py", line 106, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_49c604c9.py", line 45, in run_trial
    S_k_C = matrix_power(C, k)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_49c604c9.py", line 34, in matrix_power
    result = matrix_multiply(result, A)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_49c604c9.py", line 42, in matrix_multiply
    result[i][j] += A[i][l] * B[l][j]
                    ~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during execution, preventing complete validation of the conjecture across all seeds. | next: Fix the matrix index error in test_49c604c9.py and re-run with sufficient seeds to meet the 80% support threshold.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	68157
2	preregistration	ollama_remote	qwen3:8b	0	0	24315
3	novelty	ollama_remote	qwen3:8b	0	0	22090
4	novelty	ollama_remote	qwen3:8b	0	0	15841
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18163
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12030
7	judge	ollama_remote	qwen3:8b	0	0	17958

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 178553 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/2e8cd8937a3c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2e8cd8937a3c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2e8cd8937a3c.tar.gz` (if generated)